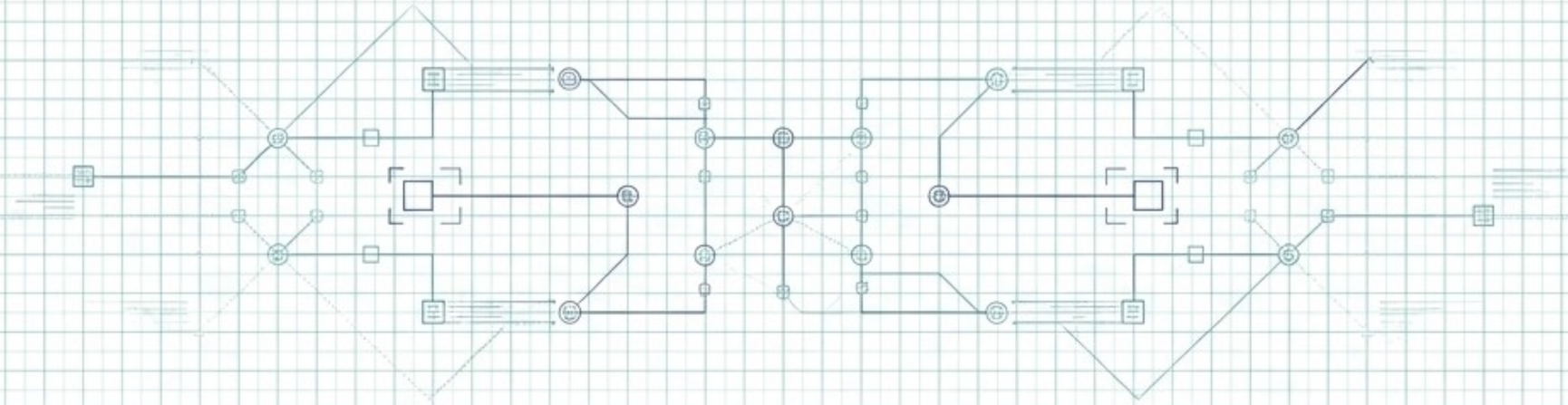


Grado en Ingeniería Informática | Dobles Grados en Informática y ADE / Matemáticas

Práctica Final: Sistema de Gestión de Matrículas Universitarias

Estructuras de Datos

DECSAI

Dpto. Ciencias de la Computación e Inteligencia Artificial (DECSAI)
Universidad de Granada

Creado por NotebookLM a partir del material docente elaborado por Joaquín Fdez-Valdivia y Rosa Rodriguez-Sánchez

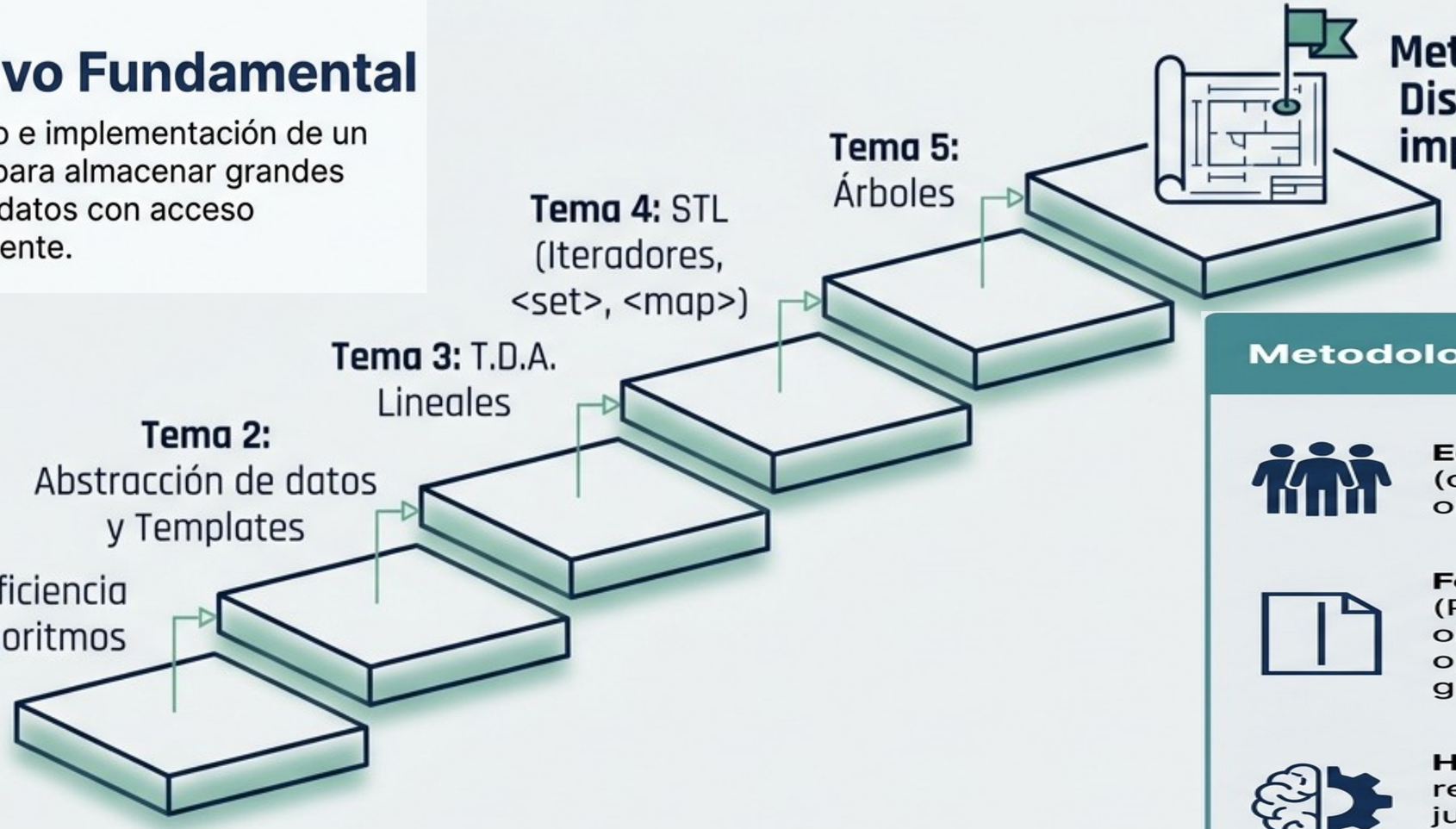
NotebookLM

Introducción: La Base Técnica del Proyecto

Integración de los conocimientos del cuatrimestre para resolver un problema real

El Objetivo Fundamental

Análisis, diseño e implementación de un **proyecto real** para almacenar grandes volúmenes de datos con acceso altamente eficiente.



Metodología de Trabajo

Equipo: 3 personas (defensa conjunta obligatoria).

Formato: Abierto (Parte básica obligatoria + Parte opcional única por grupo).

Herramientas: Uso responsable y justificado de LLMs permitido.

NotebookLM

Objetivo General: Análisis, Diseño e Implementación



Escalabilidad y Eficiencia

- Manejo de grandes volúmenes de datos.
- Uso de estructuras de acceso rápido.



Trabajo en Equipo

- Diseño y coordinación en grupos de 3 personas.
- Formato abierto: aportación de nuevas funcionalidades opcionales.

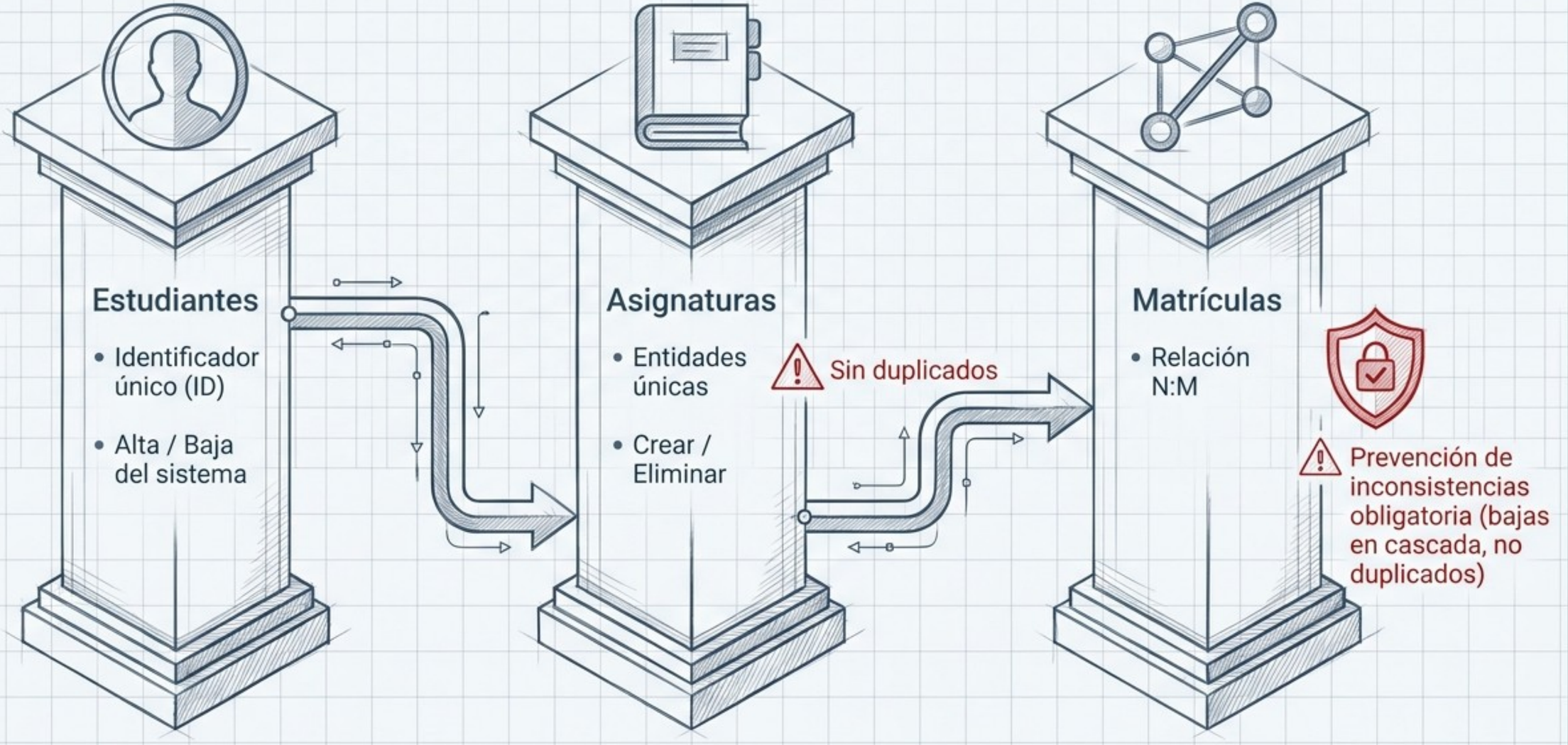


Tecnología Actual

- Integración responsable y justificada de LLMs.
- Combinación de parte básica obligatoria y desarrollo libre.

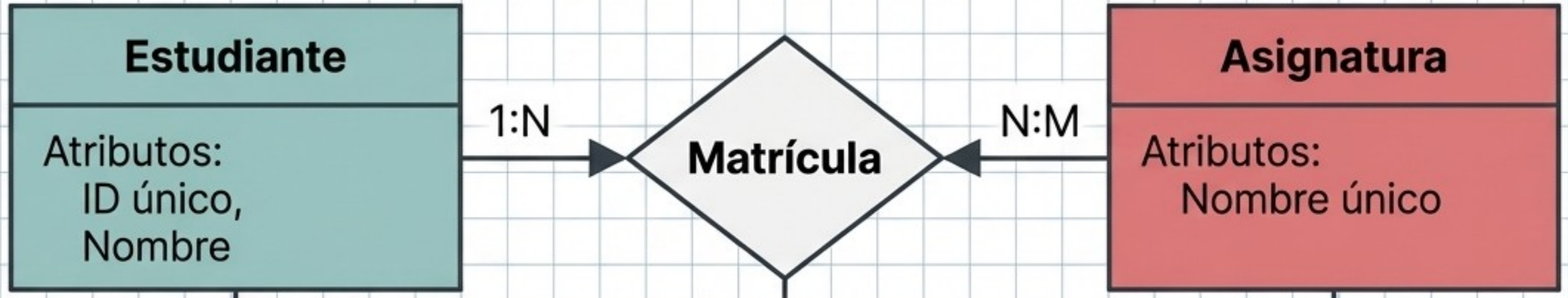
NotebookLM

Arquitectura del Sistema: Requisitos y Restricciones



NotebookLM

Arquitectura Relacional del Sistema



Regla 1: Un estudiante puede tener múltiples asignaturas.

Regla 2: Una asignatura aloja a múltiples estudiantes.

Regla 3: Cero inconsistencias (prohibido matricular al mismo estudiante dos veces en una asignatura).

NotebookLM

¿Qué debe permitir el sistema?

1.	Dar de alta y de baja estudiantes.
2.	Crear y eliminar asignaturas.
3.	Matricular y desmatricular estudiantes.
4.	Listar estudiantes (ordenados por identificador).
5.	Listar asignaturas existentes.
6.	Consultar asignaturas de un estudiante.
7.	Funcionalidades extra opcionales.

Restricciones Arquitectónicas Inquebrantables

IDs Únicos: Cada estudiante tiene un ID único (entero o cadena).

Sin Duplicados: Las asignaturas no pueden duplicarse en el sistema.

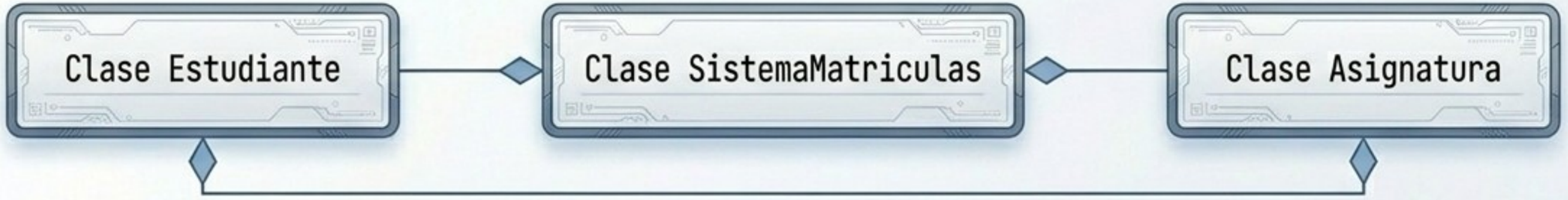
Relación N:M: Un estudiante puede tener múltiples asignaturas; una asignatura múltiples estudiantes.

Consistencia de Datos: Evitar estrictamente matrículas duplicadas o inconsistencias al borrar registros.

NotebookLM

Tablero de Control de Calidad del Software (QA): Arquitectura, Pipeline y Pruebas

Arquitectura



Pipeline de Calidad

- Inicialización explícita de contenedores
- Menú interactivo CLI con control de errores
- Modularidad y comentarios internos
- Documentación externa (Doxygen)

Matriz de Pruebas

- ☑ Matricular a un estudiante ya matriculado.
- ☑ Baja de estudiante con matrículas activas (Cascada).
- ☑ Consultar datos fantasma (inexistentes).
- ☑ Generar ficheros ficticios de prueba.
- ☑ Análisis de eficiencia teórica (inserción/búsqueda).

NotebookLM

Estructura de Clases Obligatoria

class Estudiante

```
int id
string nombre
```

class Asignatura

```
(Lógica de la asignatura)
```

class SistemaMatriculas

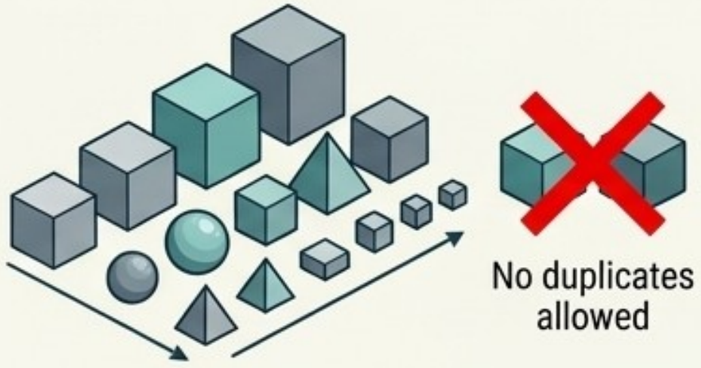
```
altaEstudiante()
bajaEstudiante()
crearAsignatura()
eliminarAsignatura()
matricular()
desmatricular()
listarEstudiantes()
listarAsignaturas()
estudiantesPorAsignatura()
```

NotebookLM

El Motor del Sistema: Contenedores STL Obligatorios

El Pilar del <set>

(Conjuntos Únicos y Ordenados)



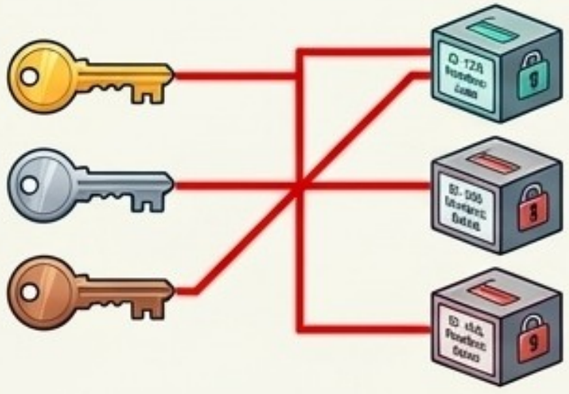
No duplicates allowed

- **Asignaturas Globales**: Almacenamiento central sin duplicados.
- **Listas de Clase**: Conjuntos de estudiantes matriculados en una asignatura.

Regla de Oro: Obligatorio definir el operador < y justificar detalladamente el criterio de ordenación.

El Pilar del <map>

(Asociación Eficiente Clave-Valor)



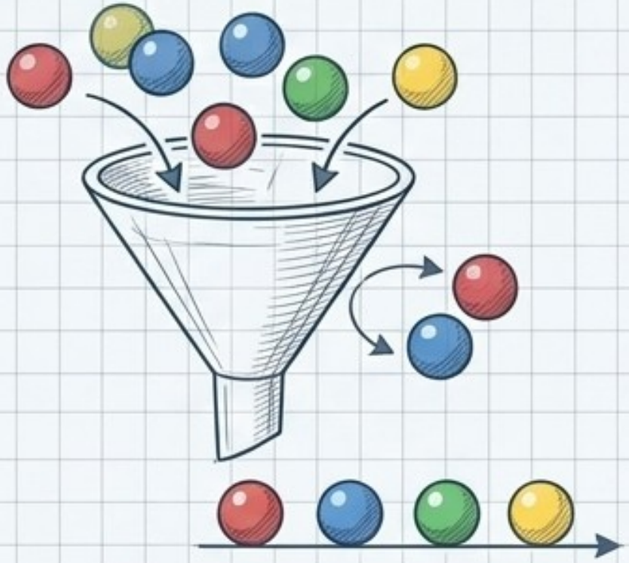
- **Directorio Principal**: Asocia el ID único con la información del Estudiante.
- **Gestor de Matrículas**: Asocia el nombre de la asignatura con el <set> de sus estudiantes.

Regla de Oro: Obligatorio gestionar búsquedas eficientes y manejar correctamente los casos donde una clave no exista.

NotebookLM

Matriz Estratégica STL: set vs map

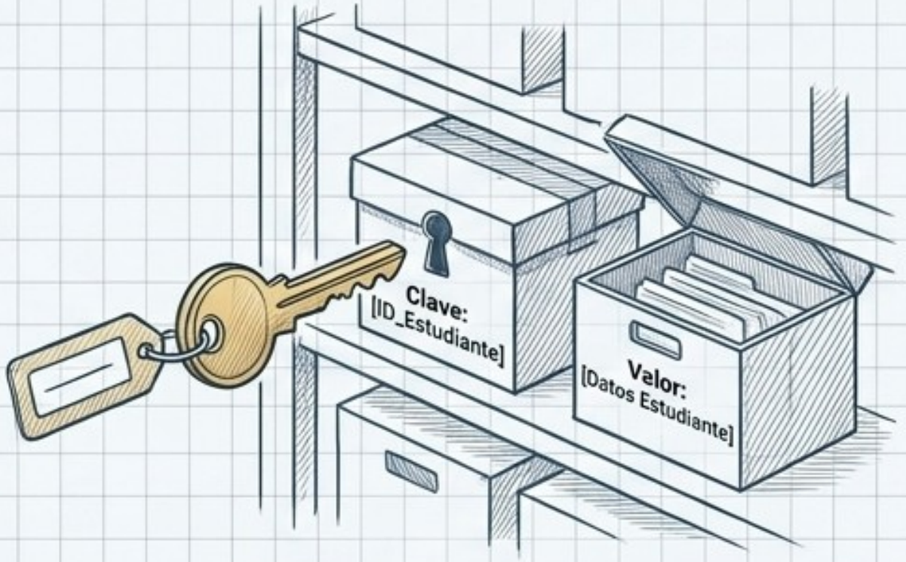
Contenedor <set>



- Almacena asignaturas únicas y conjuntos de IDs matriculados.
- Garantiza unicidad y ordenamiento automático.
- Requiere sobrecargar **operator<** explícitamente.

Implementación: Red-Black Tree

Contenedor <map>



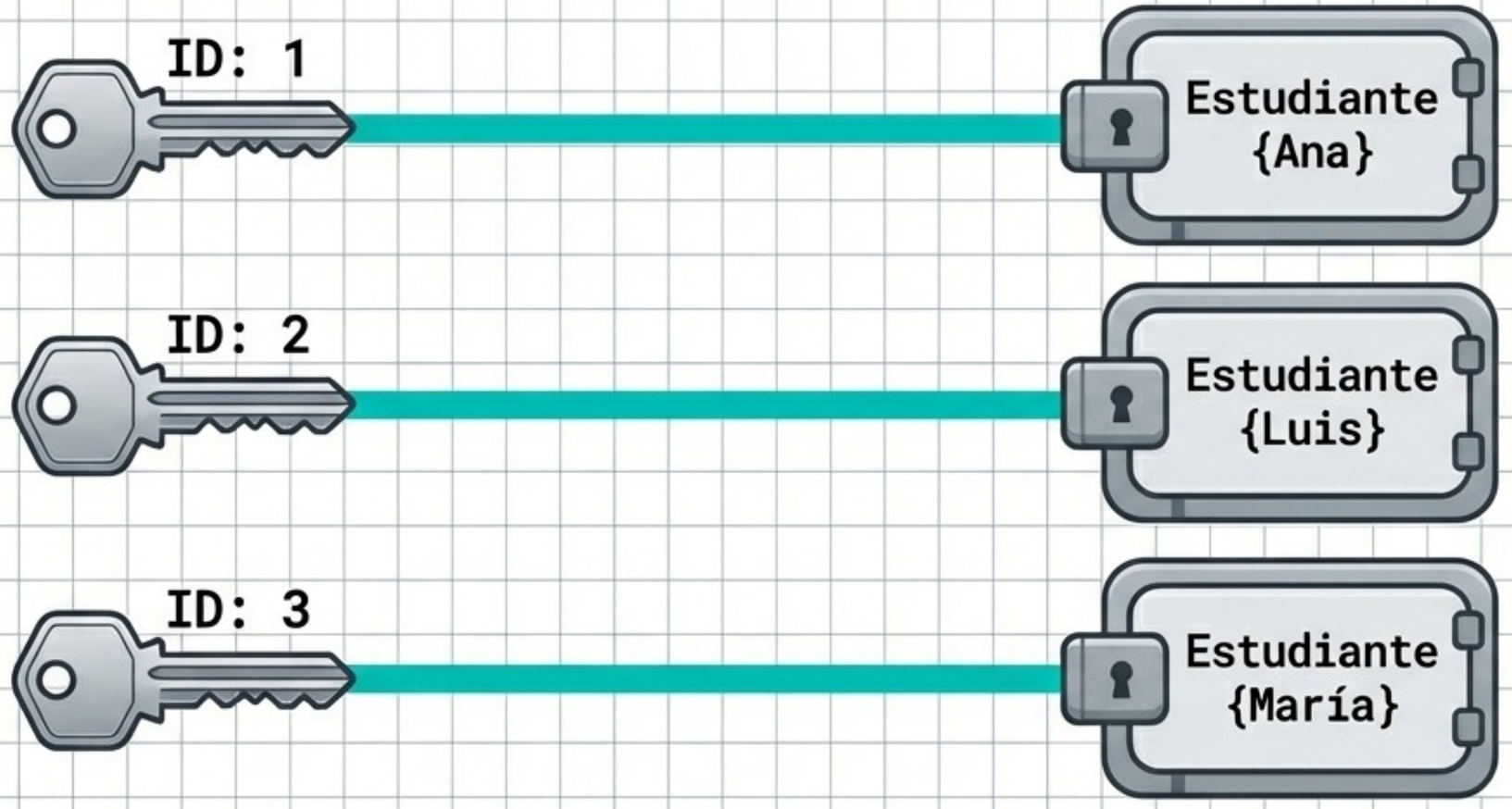
- Búsquedas eficientes estructuradas por clave.
- Mapea ID -> Estudiante.
- Mapea Asignatura -> set de IDs matriculados.

Implementación: Red-Black Tree

Eficiencia Teórica Obligatoria: Ambos contenedores garantizan una complejidad de O(log n) para operaciones de inserción y búsqueda.

NotebookLM

El Puente Clave-Valor: Acceso Directo con <map>




```
map<int, Estudiante> estudiantes;
```

Requisitos de Implementación

- ☑ - Asociar ID del estudiante con su información.
- ☑ - Gestionar búsquedas eficientes por clave.
- ☑ - **Crítico**: Manejar correctamente los casos donde la clave no existe en el mapa.

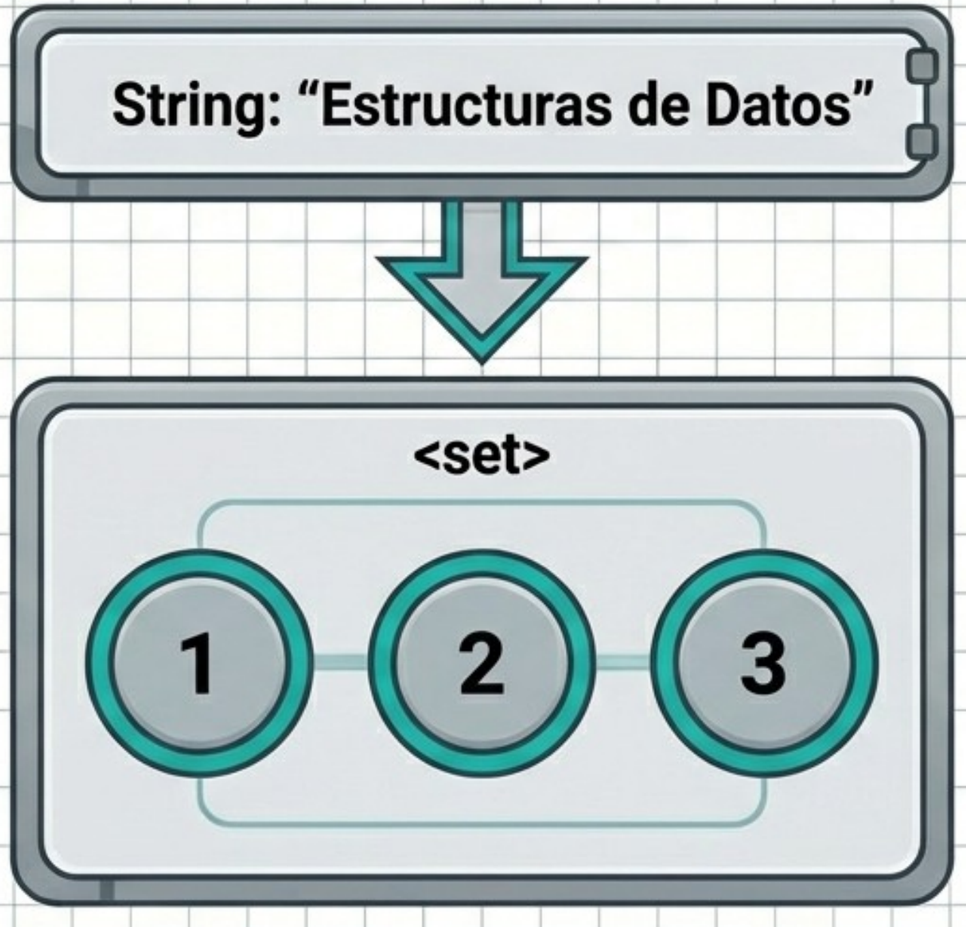
NotebookLM

El Motor del Sistema: Resolviendo la Relación N:M

```
map<string, set<int>> matriculas;
```

Por qué un <map>?

Asocia una asignatura asignatura específica con su población estudiantil en tiempo O(log N).

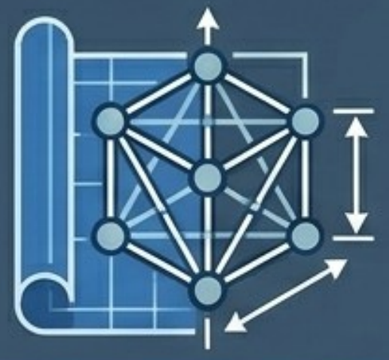


Por qué un <set>?

Garantiza inherentemente la unicidad (sin matrículas duplicadas) y mantiene los IDs ordenados. Requiere definir el operador <.

NotebookLM

Arquitectura, Calidad y Análisis



1. Diseño de Clases (Mínimo Estructural)

- Clase Estudiante
- Clase Asignatura
- Clase SistemaMatriculas

Nota: Justificar cada estructura de datos empleada.



2. Estándares de Implementación

- Menú interactivo por consola y control estricto de errores de entrada.
- Inicialización segura de contenedores.
- Código modular, legible y exhaustivamente documentado (uso de Doxygen y comentarios por módulo).



3. Análisis de Eficiencia Obligatorio

- Analizar costes de inserción/búsqueda en <set> y <map>.
- Justificar el impacto directo de estas estructuras en la escalabilidad del sistema real.

NotebookLM

Matriz de Contenedores: Obligatorios vs. Opcionales

Estructura	Complejidad	Ordenación	Caso de Uso en Proyecto
<map> / <set>	O(log N)	Ordenado por clave	Core del sistema (identificadores, matrículas)
<unordered_map>	O(1) promedio	Sin orden garantizado	Búsquedas hiper-rápidas donde el orden no importa
<vector> / <list>	O(N) búsqueda	Orden de inserción	Historiales cronológicos, colas de espera

Atención: El diseño de la funcionalidad opcional requiere justificar técnicamente la elección de nuevas estructuras.

NotebookLM

El Reto: Estructuras de Datos Opcionales

1. La Misión

Diferenciador de Grupo: Añadir nuevos módulos funcionales justificados (Ej: Estadísticas de matrículas, Gestión de expedientes).



<unordered_map>

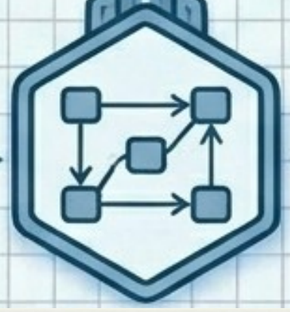


<vector>

Nuevos Módulos

2. El Requisito Técnico

Implementar estructuras de datos distintas a <set> y <map>.
Arsenal disponible: <unordered_map>, <vector>, <list>, <deque>, <queue>...



3. Criterio de Evaluación

A mayor variedad de estructuras y mejor justificación algorítmica para el nuevo módulo, mayor calificación en la parte opcional.

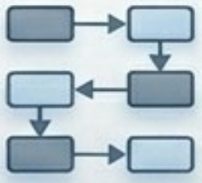
Ideas de aplicación: Estadísticas avanzadas, gestión de expedientes completos, sistema de turnos de matriculación.

NotebookLM

Maximizando la Nota: La Arquitectura Opcional

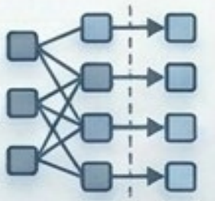
El reto es idear y justificar nuevos módulos usando estructuras estrictamente diferentes a <set> y <map>. A mayor variedad y mejor justificación técnica, mayor será la calificación.

Historial Académico



<vector> / <list>
(Registros históricos ordenados)

Módulo de Estadísticas



<unordered_map>
(Tablas Hash para búsquedas O(1))

Funcionalidad Extra (+0.5 Pts)

Colas de Espera



<deque> / <queue>
(Gestión de turnos de matriculación)

<deque> o <list>

Registro de auditoría o sistema 'Undo' para las últimas acciones realizadas.

NotebookLM

Matriz de Control de Calidad (Casos de Prueba)

Acción del Usuario	Protección del Sistema
 Matricular un estudiante ya matriculado.	 Prevención de duplicidad sin corromper datos.
 Eliminar un estudiante con matrículas activas.	 Cascading delete (baja en todas sus asignaturas).
 Consultar información inexistente.	 Control de excepciones, evitar segmentation faults.
 Solicitar listados.	 Retorno completo y estrictamente ordenado.

Requisito: Crear ficheros ficticios de estudiantes y asignaturas para poblar y probar estos escenarios antes de la entrega.

NotebookLM

Estándares de Implementación y UX



Pilar 1: Interfaz

Menú interactivo por consola. Control robusto de errores en la entrada de datos.



Pilar 2: Modularidad

Código legible, distribuido lógicamente en ficheros separados.



Pilar 3: Seguridad

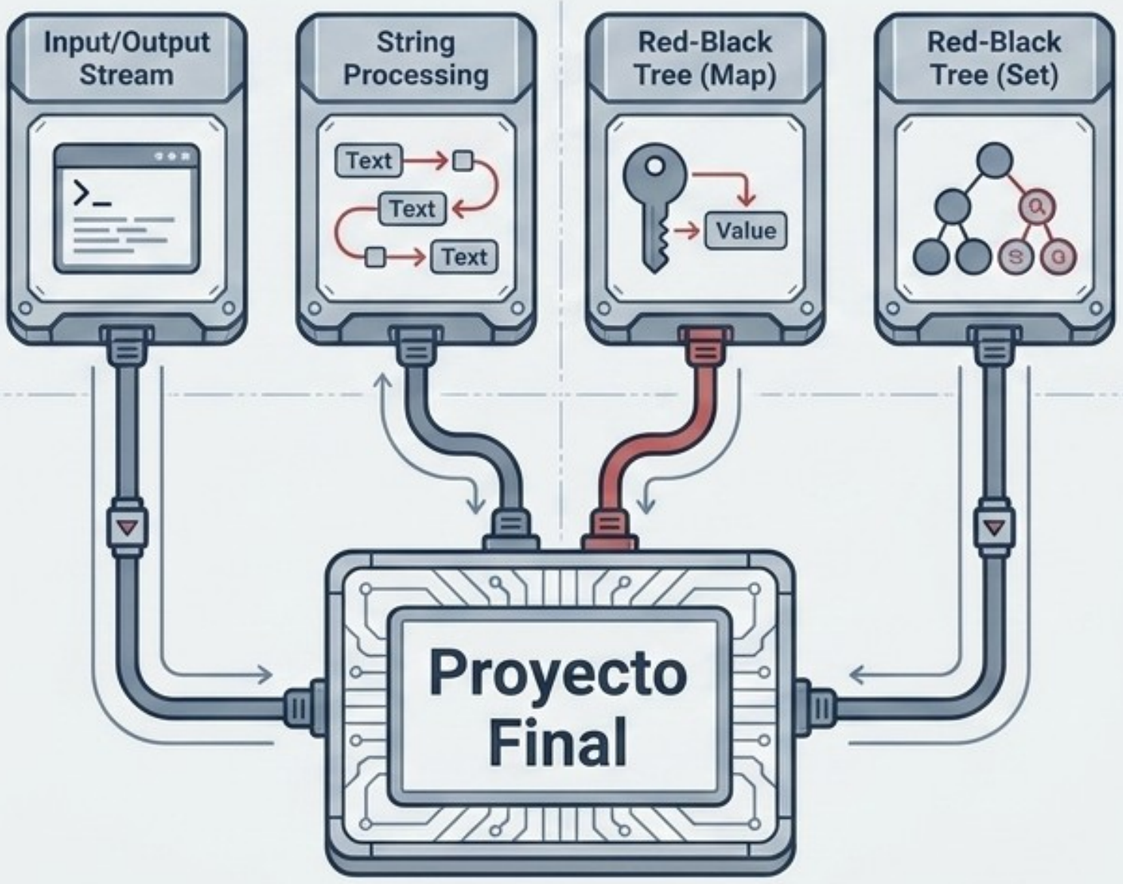
Inicialización correcta de todos los contenedores de la STL antes de su uso.



Pilar 4: Documentación

Comentarios explicativos por módulo y documentación completa del código usando Doxygen.

NotebookLM

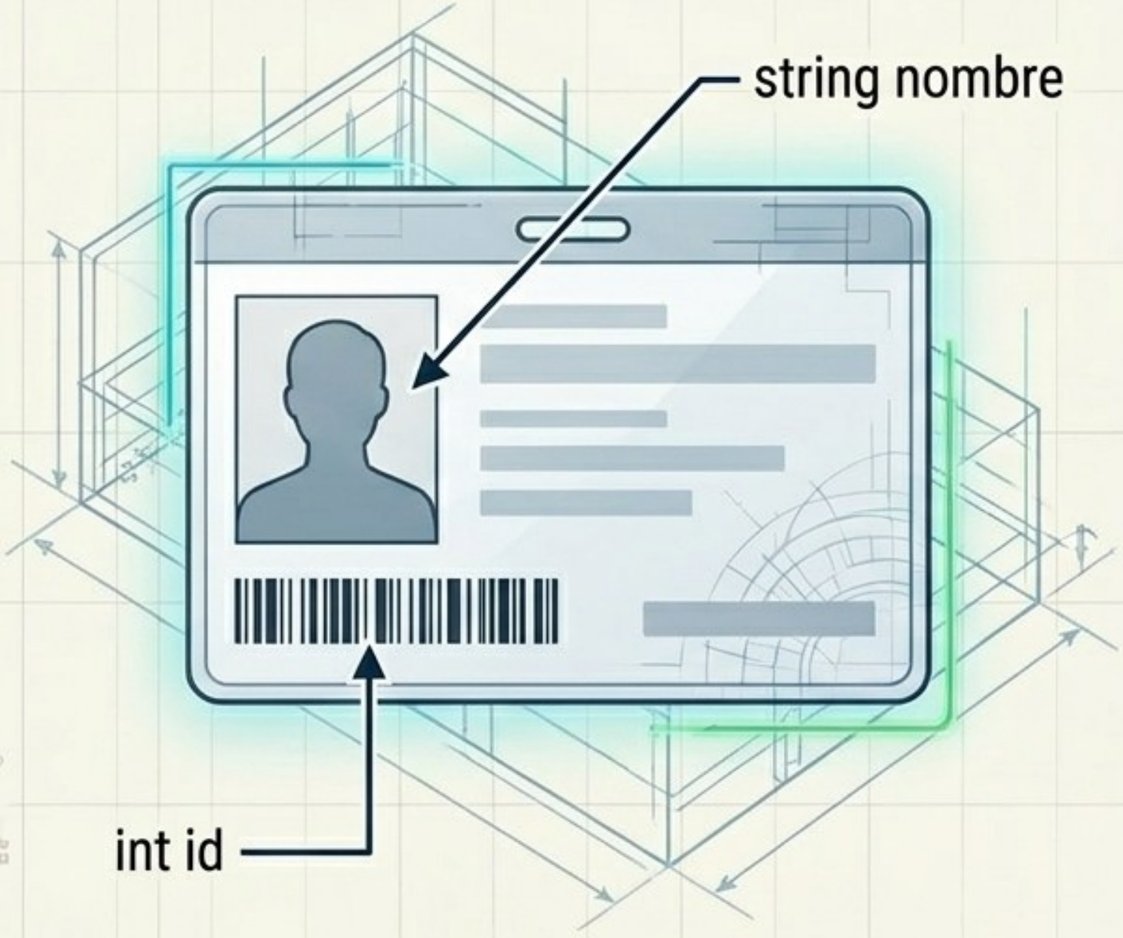



```
#include <iostream>
#include <string>
#include <map>
#include <set>

using namespace std;
```

NotebookLM

Clase Estudiante



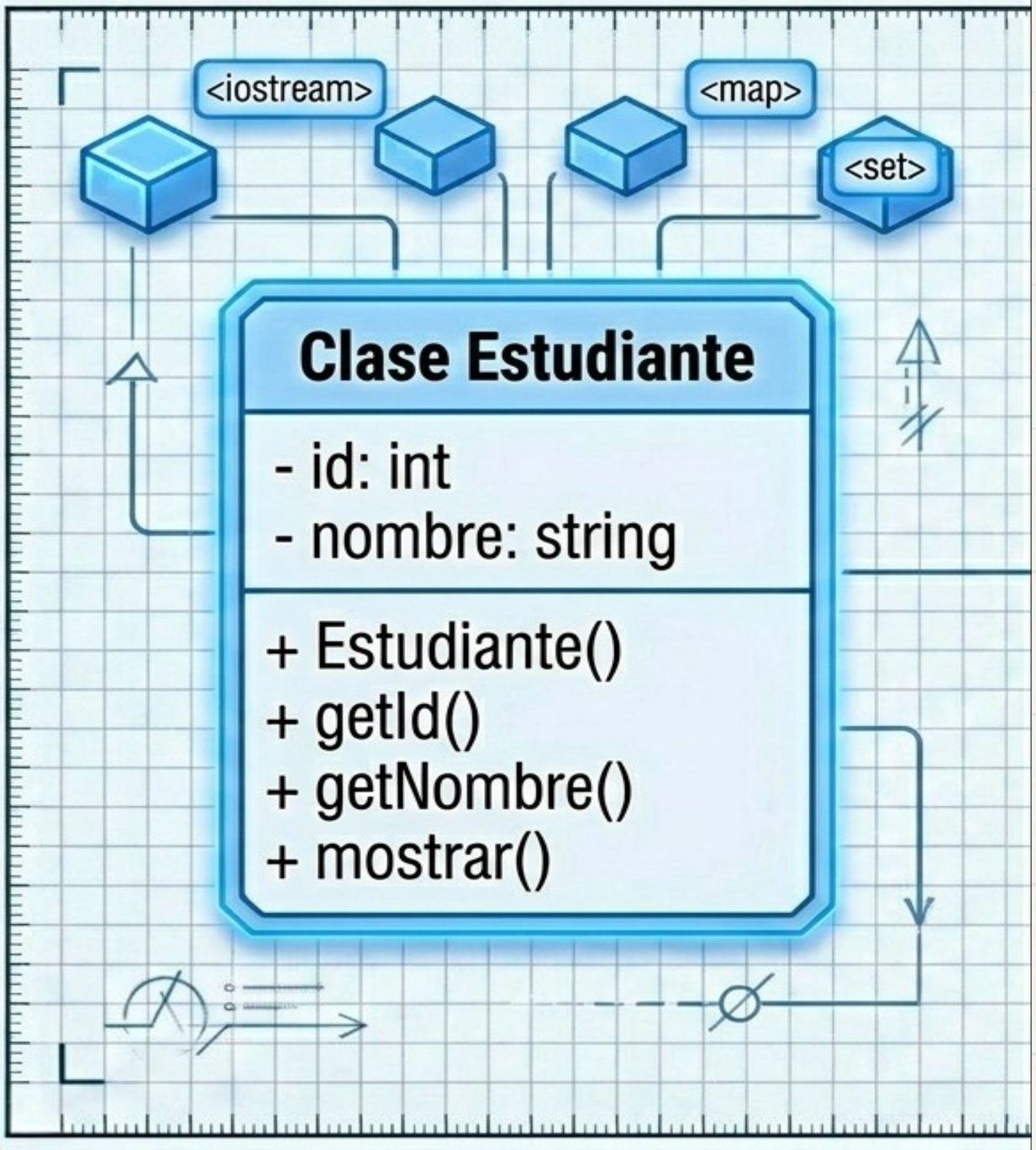

```cpp
#include <iostream>
#include <string>
#include <map>
#include <set>
using namespace std;

/* ===== Clase Estudiante ===== */
class Estudiante {
private:
    int id;
    string nombre;
public:
    Estudiante(int id, const string& nombre)
        : id(id), nombre(nombre) {}

    int getId() const { return id; }

    string getNombre() const {
        // Código
    }
    void mostrar() const {
        // Código
    }
};
```

NotebookLM



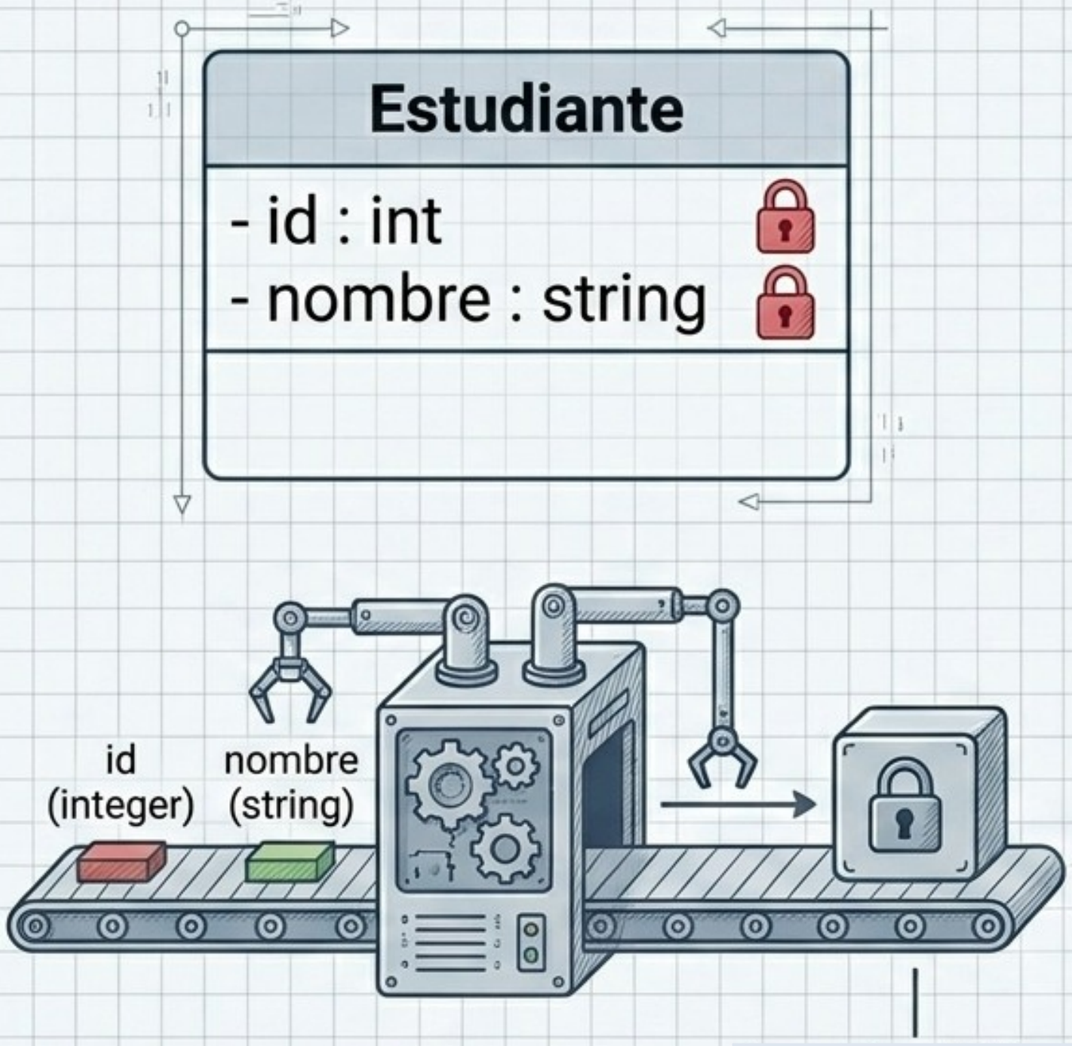

```cpp
#include <iostream>
#include <string>
#include <map>
#include <set>
using namespace std;

/* ===== Clase Estudiante ===== */
class Estudiante {
private:
    int id;
    string nombre;
public:
    Estudiante(int id, const string& nombre)
        : id(id), nombre(nombre) {}

    int getId() const { return id; }

    string getNombre() const {
        // Código
    }
    void mostrar() const {
        // Código
    }
};
```

NotebookLM



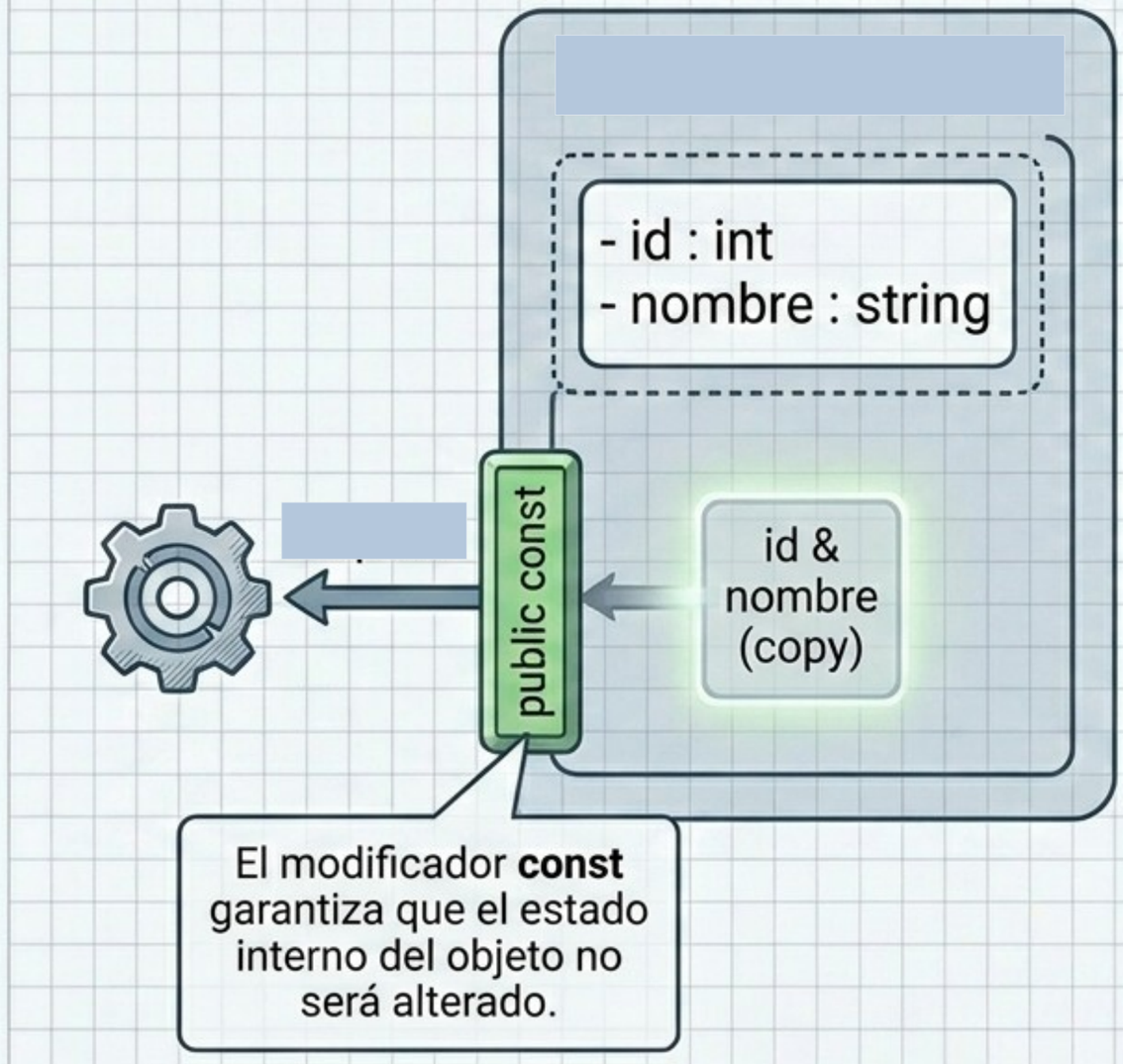

```cpp
#include <iostream>
#include <string>
#include <map>
#include <set>
using namespace std;

/* ===== Clase Estudiante ===== */
class Estudiante {
private:
    int id;
    string nombre;
public:
    Estudiante(int id, const string& nombre)
        : id(id), nombre(nombre) {}

    int getId() const { return id; }

    string getNombre() const {
        // Código
    }
    void mostrar() const {
        // Código
    }
};
```

NotebookLM




```cpp
#include <iostream>
#include <string>
#include <map>
#include <set>
using namespace std;

/* ===== Clase Estudiante ===== */
class Estudiante {
private:
    int id;
    string nombre;
public:
    Estudiante(int id, const string& nombre)
        : id(id), nombre(nombre) {}

    int getId() const { return id; }

    string getNombre() const {
        // Código
    }
    void mostrar() const {
        // Código
    }
};
```

NotebookLM




```cpp
/* === Sistema de Matrículas === */
class SistemaMatriculas {
private:
   // ID -> Estudiante
   map<int, Estudiante> estudiantes;

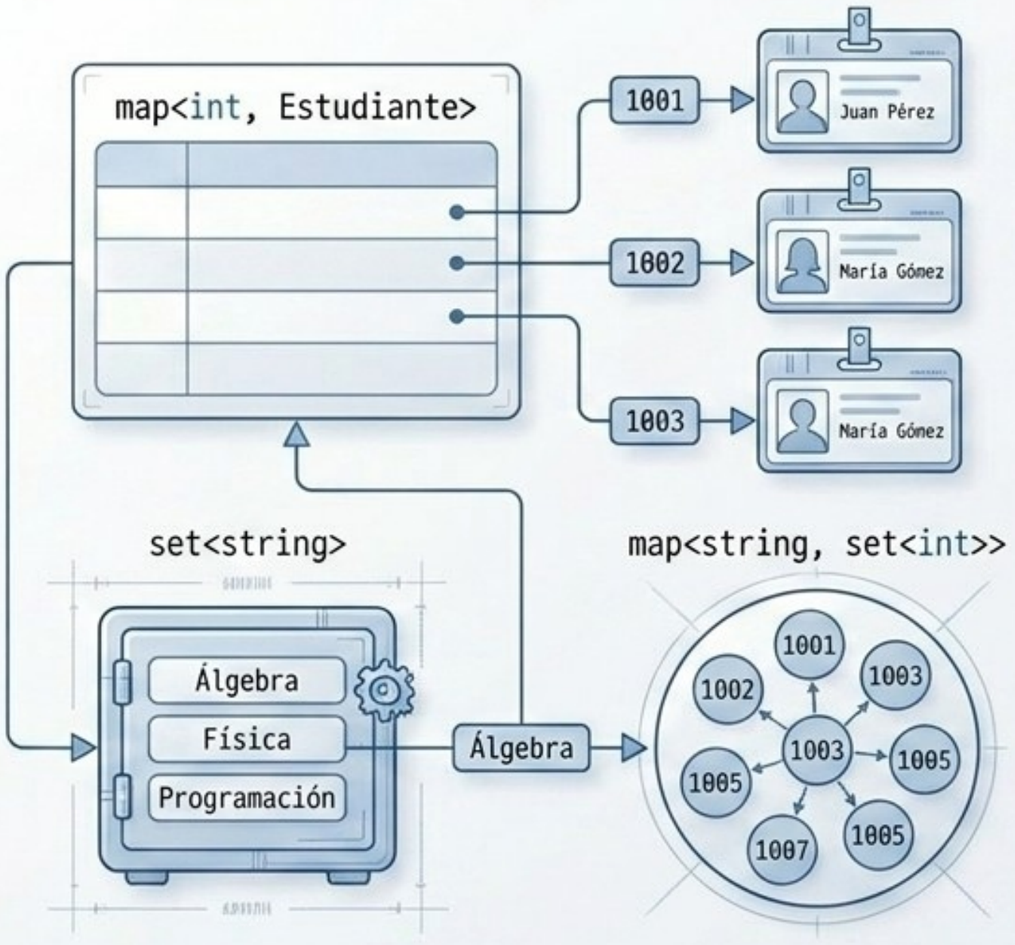
   // Asignaturas únicas
   set<string> asignaturas;

   // Asignatura -> IDs estudiantes
   map<string, set<int>> matriculas;

public:
  /* Módulos de gestión a continuación... */
};
```

NotebookLM

Modelo Mental: Bases de Datos en Memoria



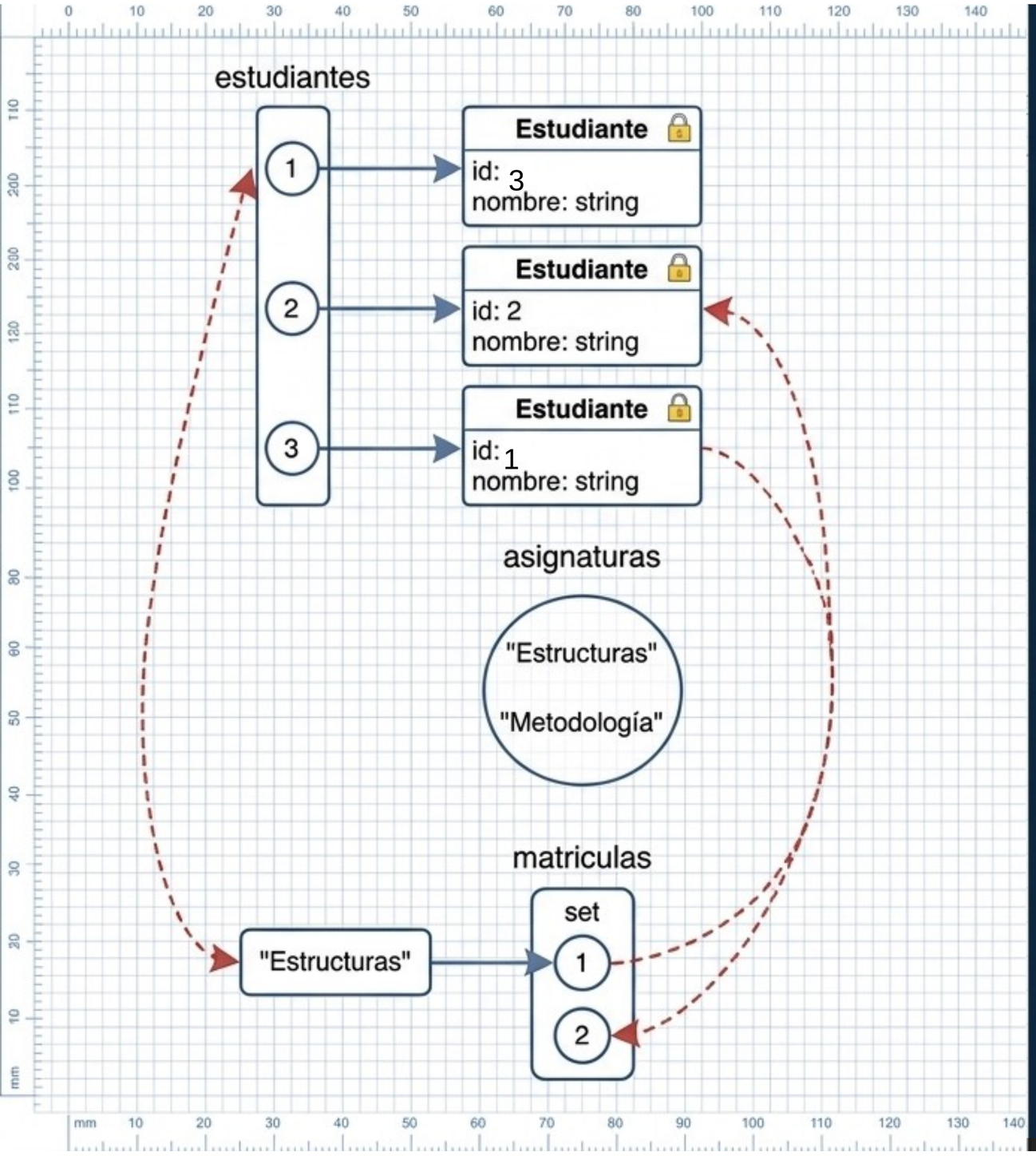

```
/* === Sistema de Matrículas === */
class SistemaMatriculas {
private:
   // ID -> Estudiante
   map<int, Estudiante> estudiantes;

   // Asignaturas únicas
   set<string> asignaturas;

   // Asignatura -> IDs estudiantes
   map<string, set<int>> matriculas;

public:
  /* Módulos de gestión a continuación... */
};
```

NotebookLM




```cpp
/* === Sistema de Matrículas === */
class SistemaMatriculas {
private:
   // ID -> Estudiante
   map<int, Estudiante> estudiantes;

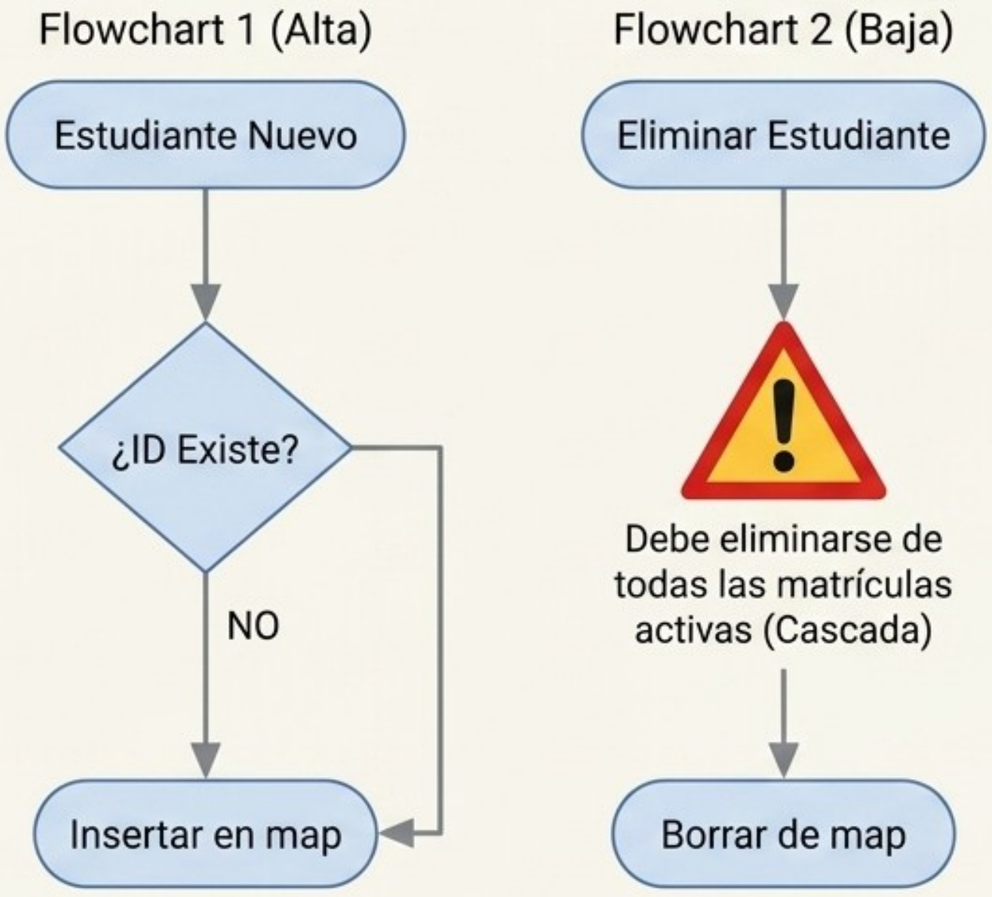
   // Asignaturas únicas
   set<string> asignaturas;

   // Asignatura -> IDs estudiantes
   map<string, set<int>> matriculas;

public:
  /* Módulos de gestión a continuación... */
};
```

NotebookLM

Modelo Mental: Gestión de Entidades y Cascada



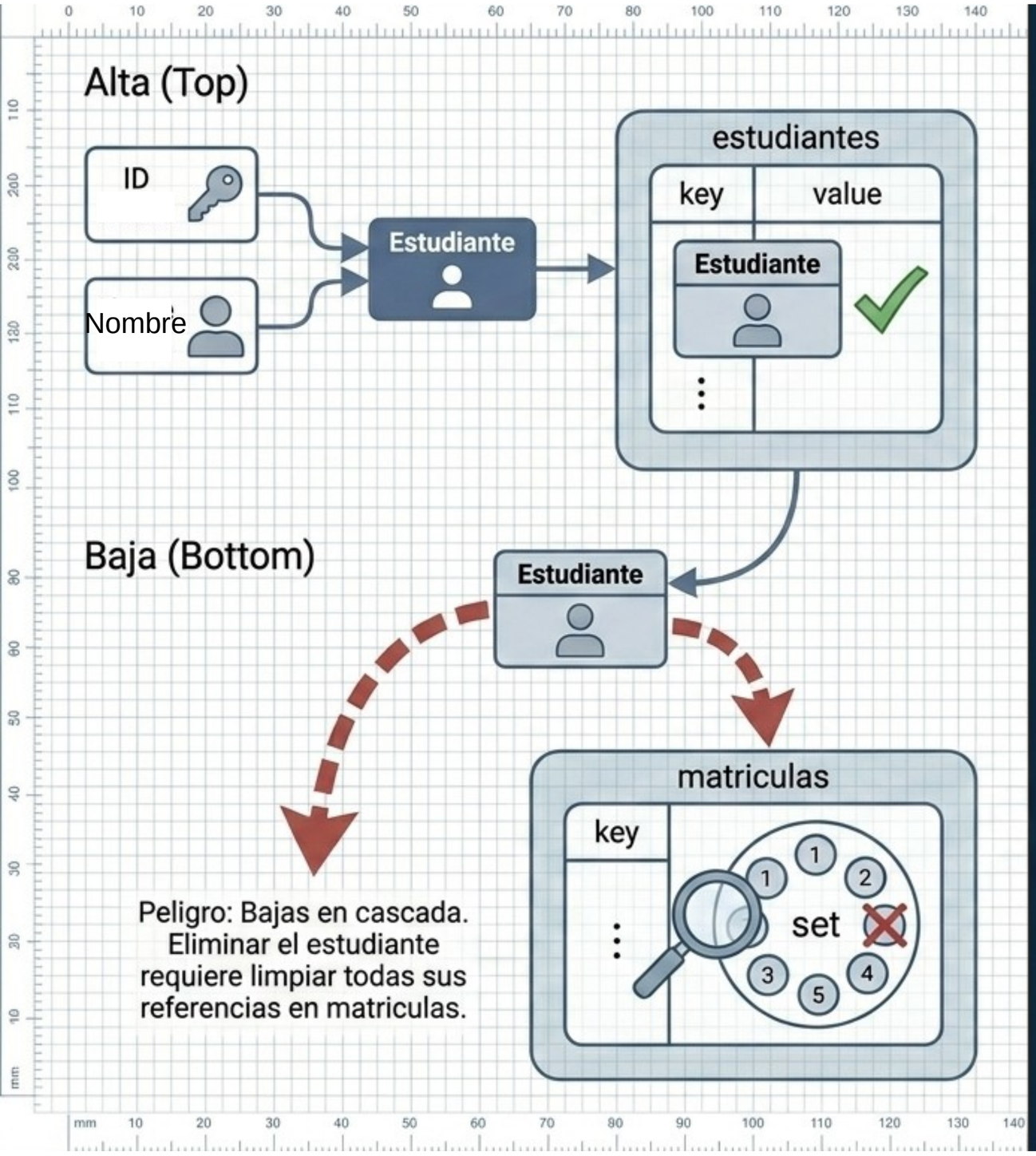

```
/* --- Gestión de estudiantes --- */
void altaEstudiante(int id, const string& nombre) {
    // Código
}

void bajaEstudiante(int id) {
    // Código
    // Debería eliminar al estudiante
    // de todas las matrículas
}


/* --- Gestión de asignaturas --- */
void crearAsignatura(const string& nombre) {
    // Código
}

void eliminarAsignatura(const string& nombre) {
    // Código
}
```

NotebookLM



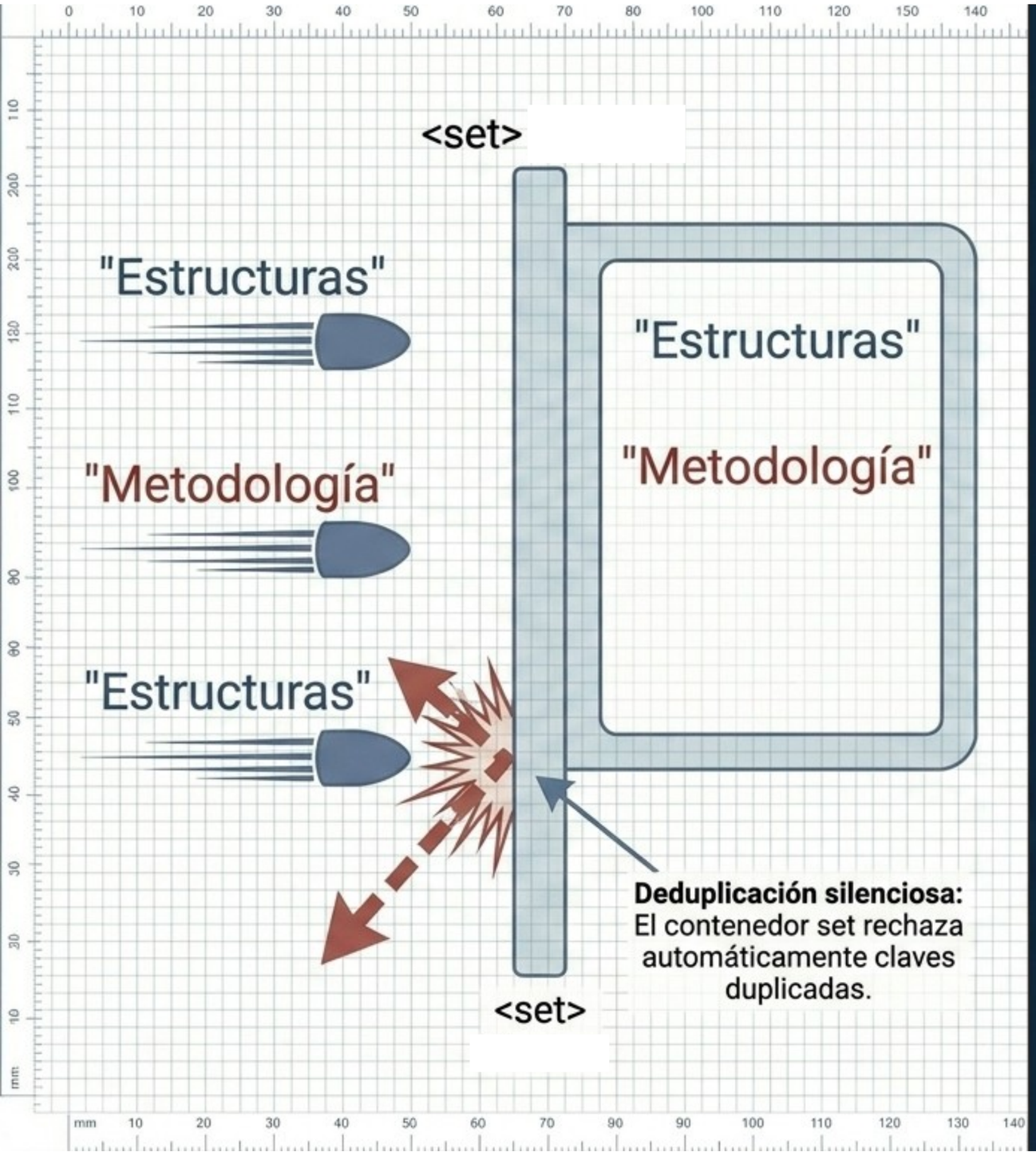

```
/* --- Gestión de estudiantes --- */
void altaEstudiante(int id, const string& nombre) {
    // Código
}

void bajaEstudiante(int id) {
    // Código
    // Debería eliminar al estudiante
    // de todas las matrículas
}


/* --- Gestión de asignaturas --- */
void crearAsignatura(const string& nombre) {
    // Código
}

void eliminarAsignatura(const string& nombre) {
    // Código
}
```

NotebookLM




```
/* --- Gestión de estudiantes --- */
void altaEstudiante(int id, const string& nombre) {
    // Código
}

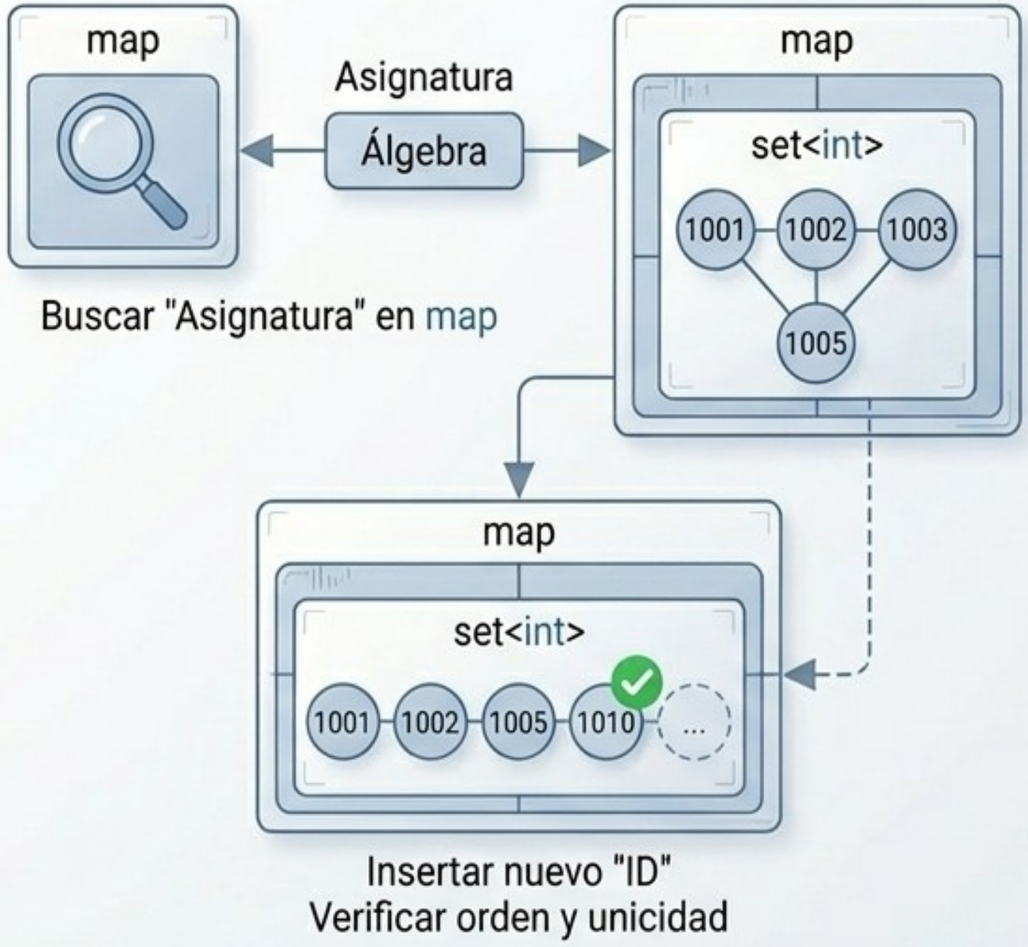
void bajaEstudiante(int id) {
    // Código
    // Debería eliminar al estudiante
    // de todas las matrículas
}


/* --- Gestión de asignaturas --- */
void crearAsignatura(const string& nombre) {
    // Código
}

void eliminarAsignatura(const string& nombre) {
    // Código
}
```

NotebookLM

Modelo Mental: Transacciones de Matrícula

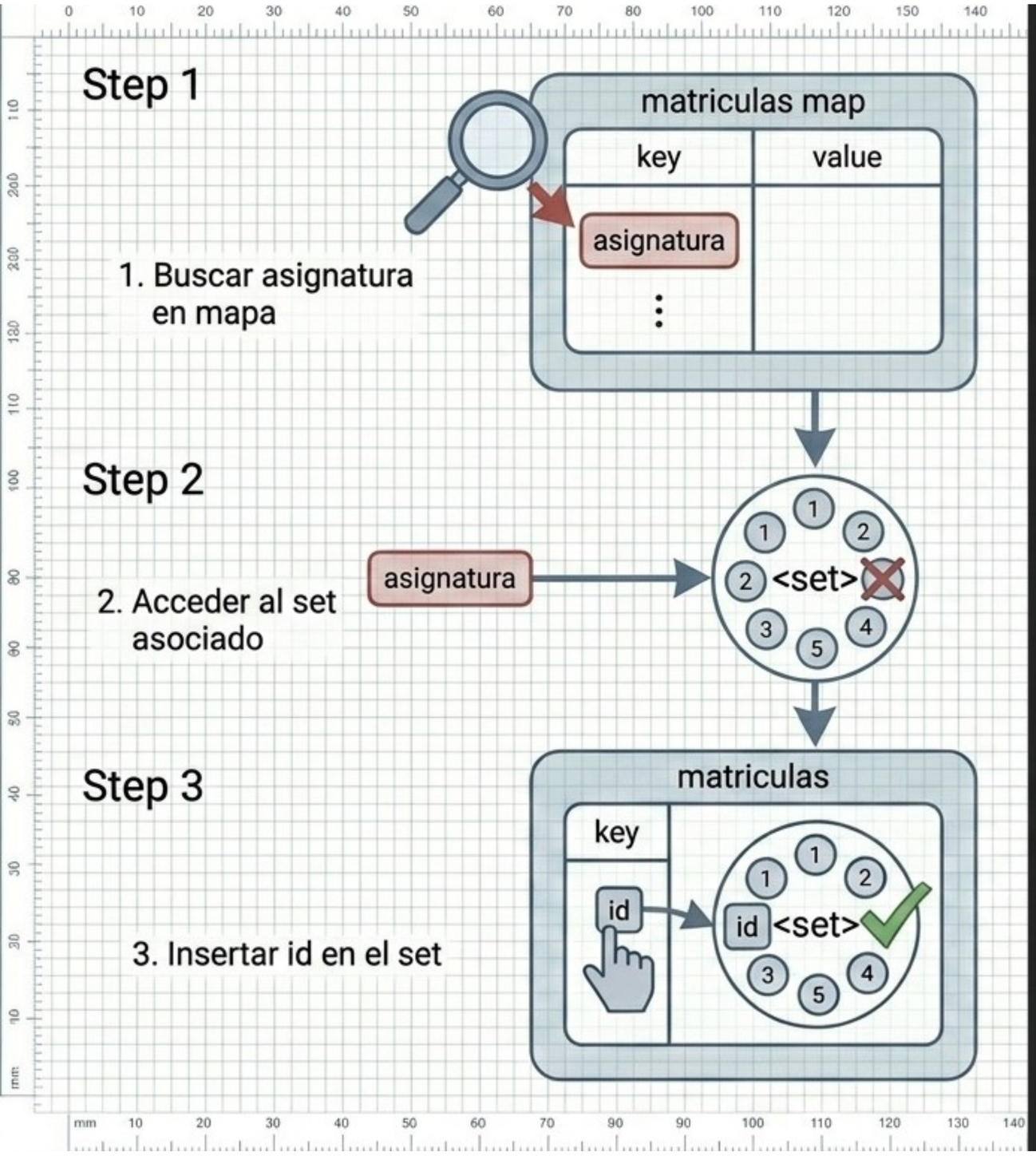



```
/* ==========================
    ----- Matrículas -----
========================== */

void matricular(int id,
    const string& asignatura) {
    //Código
}

void desmatricular(int id,
    const string& asignatura) {
    //Código
}
```

NotebookLM



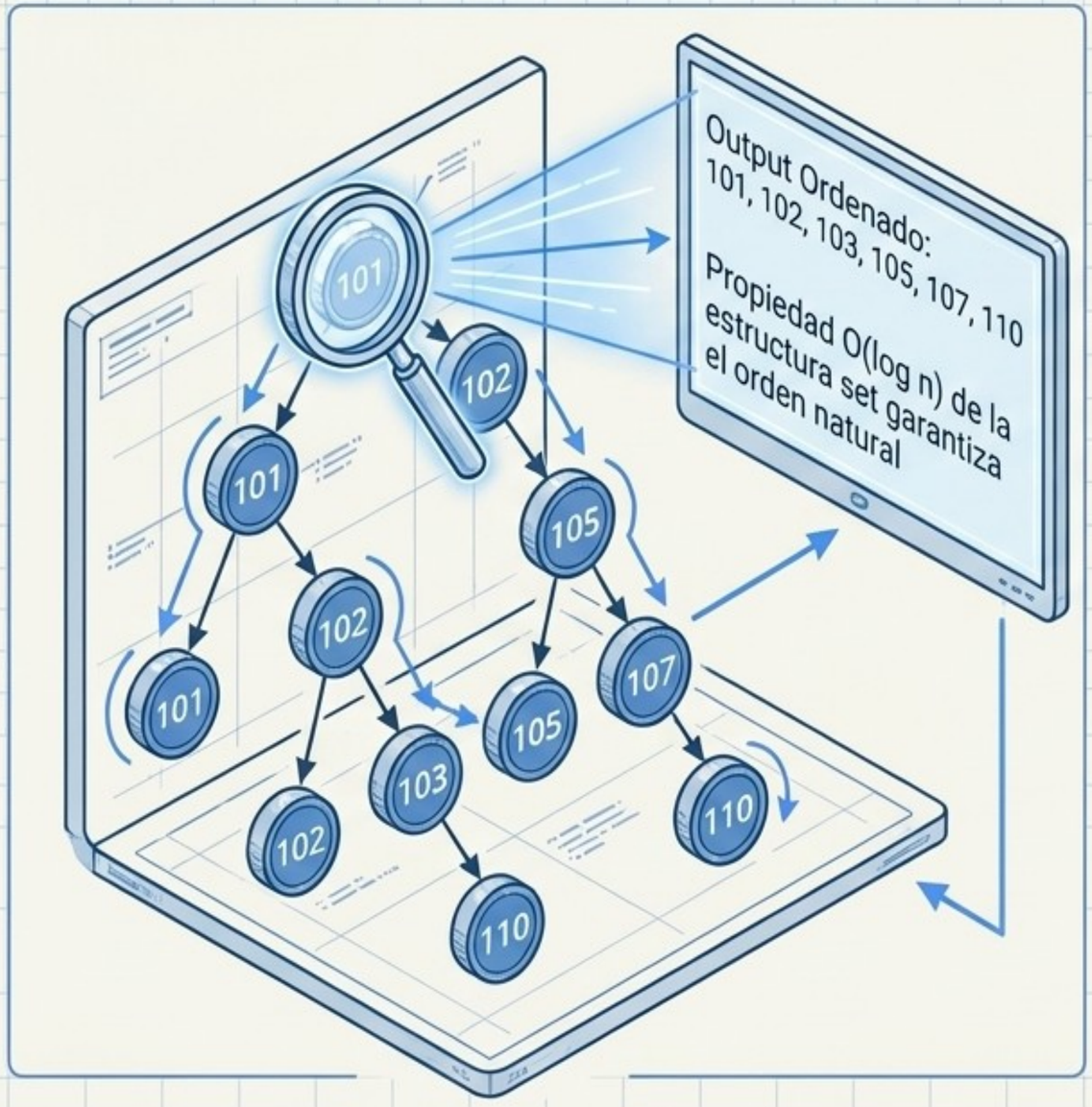

```
/* ==========================
    ----- Matrículas -----
========================== */

void matricular(int id,
    const string& asignatura) {
    //Código
}

void desmatricular(int id,
    const string& asignatura) {
    //Código
}
```

NotebookLM

Modelo Mental: Iteradores C++



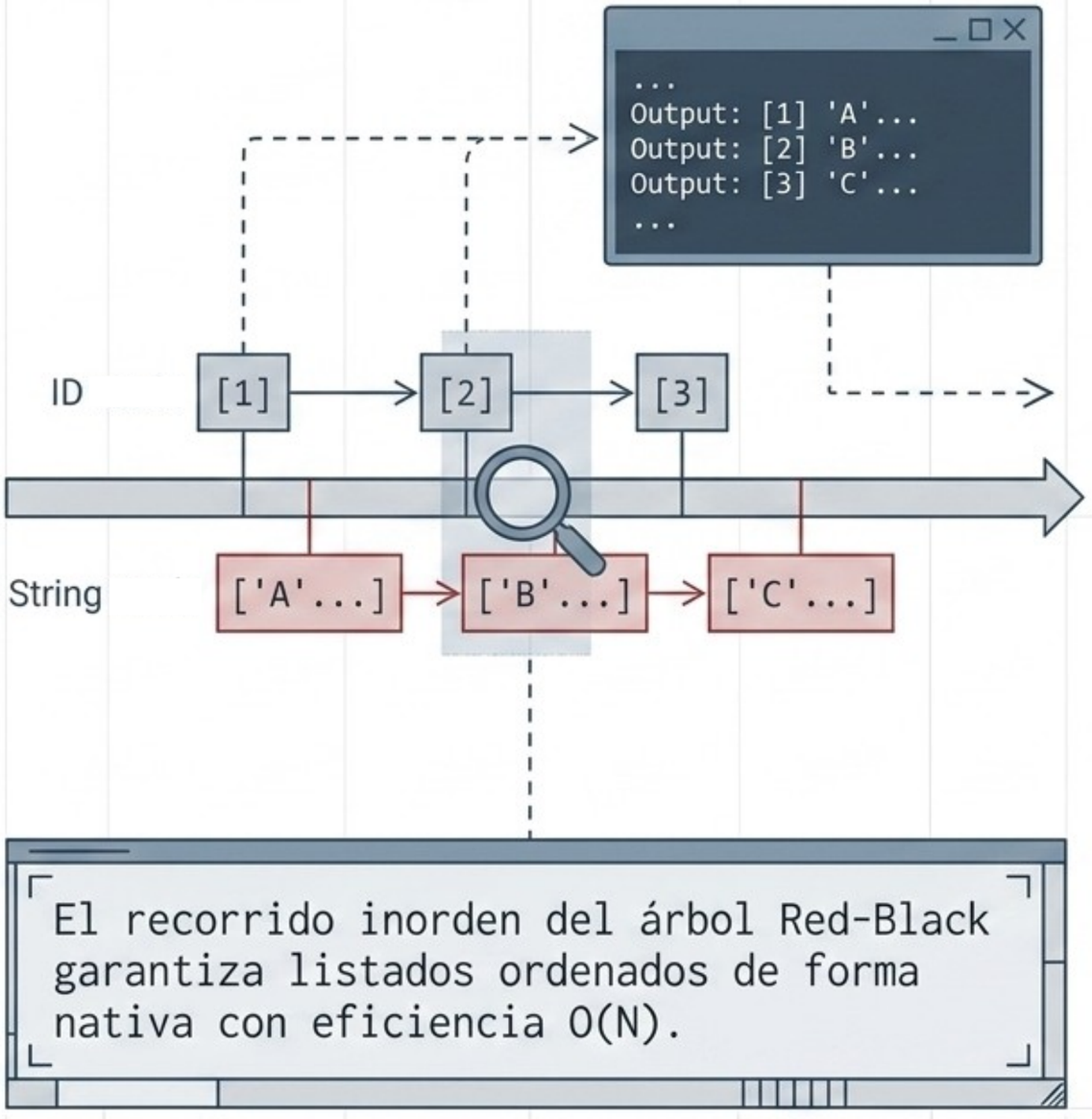

```
    /* ----- Consultas ----- */

    void listarEstudiantes() const {
        //Código
    }

    void listarAsignaturas() const {
        //Código
    }

    void estudiantesPorAsignatura(
        const string& asignatura) const {
        //Código
    }
};
```

NotebookLM




```
    /* ----- Consultas ----- */

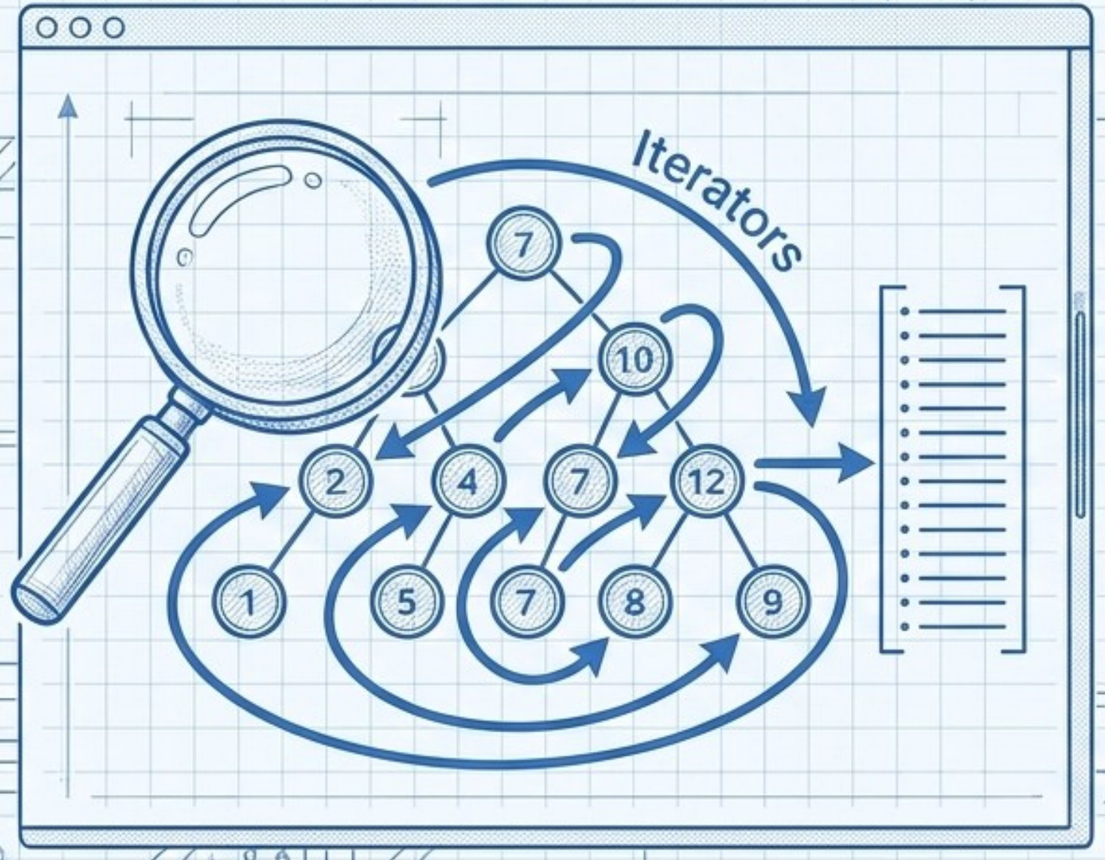
    void listarEstudiantes() const {
        //Código
    }

    void listarAsignaturas() const {
        //Código
    }

    void estudiantesPorAsignatura(
        const string& asignatura) const {
        //Código
    }
};
```

NotebookLM

Motor de Consultas -> Uso de **Iteradores STL** para recorridos eficientes y ordenados.




```
    /* ----- Consultas ----- */

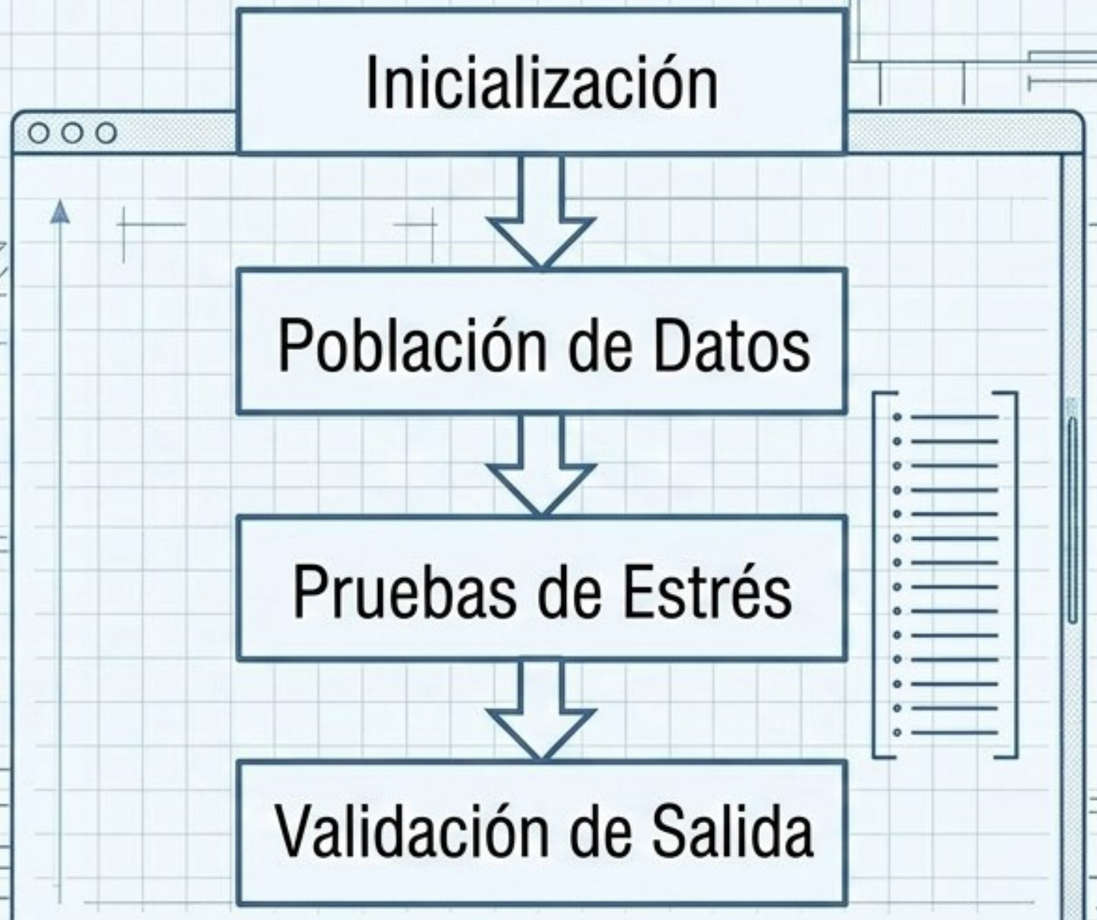
    void listarEstudiantes() const {
        //Código
    }

    void listarAsignaturas() const {
        //Código
    }

    void estudiantesPorAsignatura(
        const string& asignatura) const {
        //Código
    }
};
```

NotebookLM

Flujo de Ejecución (main)




```cpp
int main() {
    SistemaMatriculas sistema;

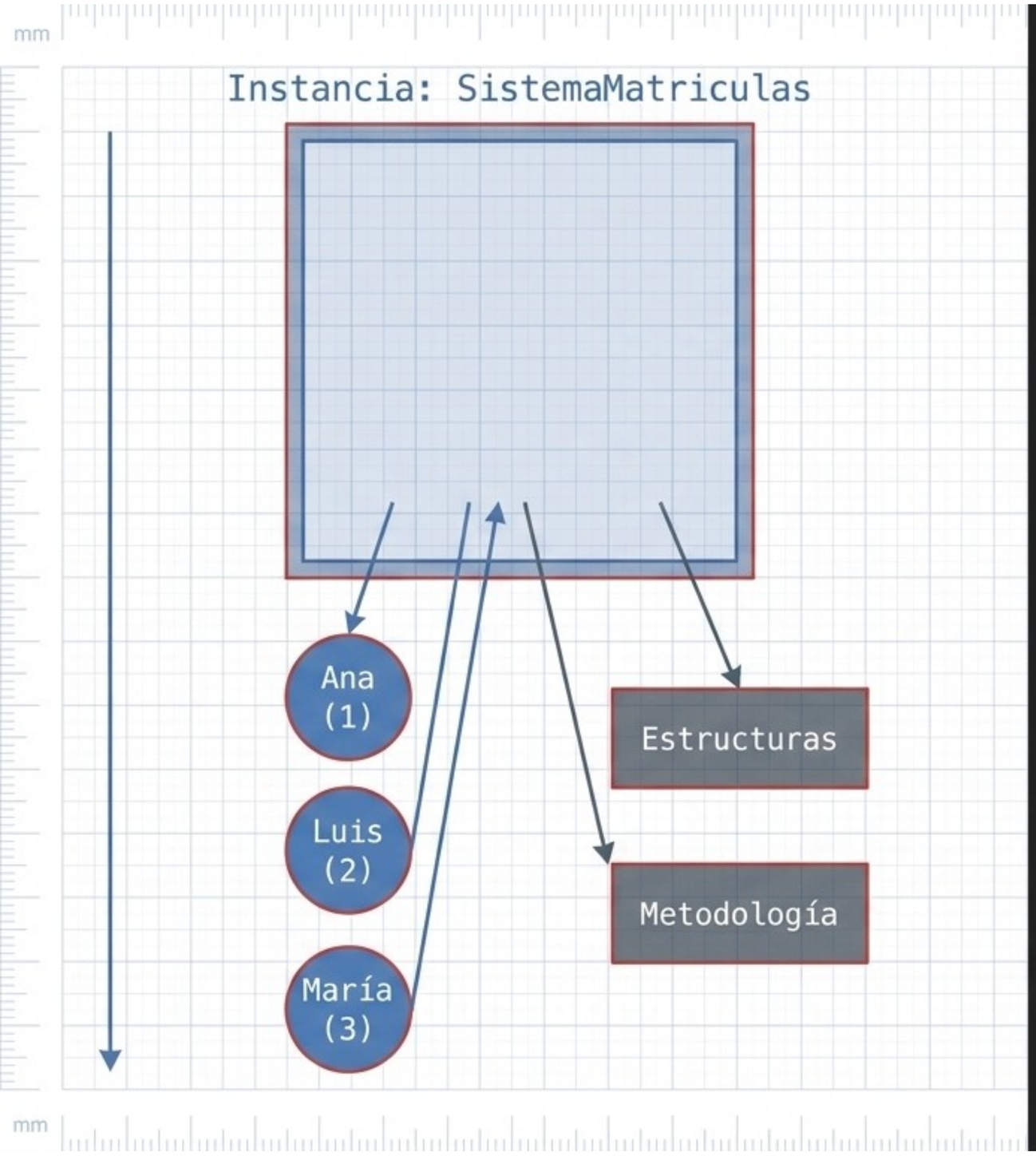
    // Datos de prueba
    sistema.altaEstudiante(1, "Ana");
    sistema.altaEstudiante(2, "Luis");
    sistema.altaEstudiante(3, "María");

    sistema.crearAsignatura("Estructuras de Datos");
    sistema.crearAsignatura("Metodología...");

    sistema.matricular(1, "Estructuras...");
    sistema.matricular(2, "Estructuras...");
    sistema.matricular(3, "Metodología...");

    sistema.listarEstudiantes();
    sistema.listarAsignaturas();
    // ...
    return 0;
}
```

NotebookLM



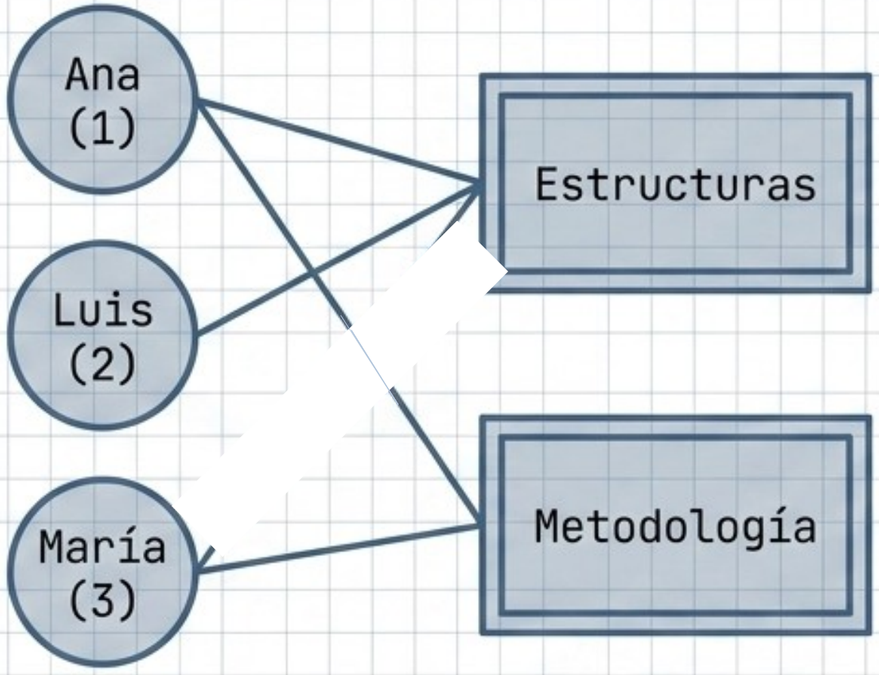

```java
/* ================================
   Programa principal
   ================================ */
int main() {
  SistemaMatriculas sistema;

  // Datos de prueba
  sistema.altaEstudiante(1, "Ana");
  sistema.altaEstudiante(2, "Luis");
  sistema.altaEstudiante(3, "María");

  sistema.crearAsignatura("Estructuras de Datos");
  sistema.crearAsignatura("Metodología de la Programación");
}
```

NotebookLM

```
> Listando estudiantes...
[1] Ana
[2] Luis
[3] María
```




```
> Estudiantes en Estructuras de Datos:
[1] Ana
[2] Luis
```

```
    sistema.matricular(1, "Estructuras de
Datos");
    sistema.matricular(2, "Estructuras de
Datos");
    sistema.matricular(3, "Metodología de la
Programación");

    sistema.listarEstudiantes();
    sistema.listarAsignaturas();
    sistema.estudiantesPorAsignatura("Estructuras
de Datos");

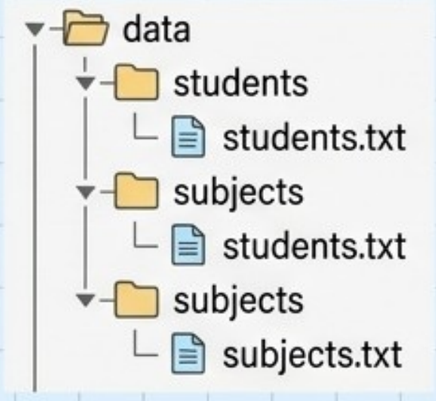
    return 0;
}
```

NotebookLM

Validación del Sistema en Entornos Reales



- **Casos de Prueba Estresantes (Obligatorios):**
  - **Duplicidad:** Intentar matricular a un estudiante que ya está matriculado.
  - **Cascada de Borrado:** Eliminar a un estudiante con matrículas activas (¿Quedan registros fantasma?).
  - **Accesos Nulos:** Consultar información inexistente (IDs o Asignaturas falsas).
  - **Integridad de Orden:** Generar listados completos y verificar su correcta ordenación.



- **Entorno de Datos (Ficheros):**
  - Diseñar ficheros ficticios de estudiantes y asignaturas.
  - Ajustar el volumen y formato a las necesidades de carga inicial del sistema de pruebas.

NotebookLM

Arquitectura de Entrega: practicafinal.tgz

- **practica_final/**
  - **src/** — Archivos fuente .cpp
  - **include/** — Cabeceras .h
  - **data/** — Ficheros ficticios
  - **Makefile** — Imprescindible para compilación
  - **readme.txt** — Documentación del equipo

```
prompt% tar zcv practicafinal.tgz
practica_final
```

ATENCIÓN: Realizar 'limpieza' previa. **PROHIBIDO** incluir ficheros objeto (.o) o ejecutables en el archivo final.

NotebookLM

Protocolo Estricto de Empaquetado y Entrega

1. **Limpieza y Estructura del Directorio:**
   - **Prohibido:** No incluir ficheros objeto (.o) ni ejecutables.
   - Distribuir en subdirectorios claros (fuentes, datos).
   - **Crítico:** Incluir el archivo Makefile para compilación automática.

3. **El Fichero readme.txt (Obligatorio):**
   - Documentar qué se ha hecho, cómo, y referencias usadas.
   - Detallar la implicación exacta de cada miembro del equipo.
   - Reportar bugs honestamente.
   - **Declaración LLM:** Justificación precisa y documentada si se ha utilizado alguna IA.

```
2. Ejecución de Empaquetado:

prompt% tar zcv practicafinal.tgz practica_final

Subir el .tgz a PRADO antes de la fecha límite.
(Obligatorio para todos los miembros).
```

NotebookLM

Anatomía del readme.txt



Metodología: Qué se ha hecho y cómo se ha hecho (incluyendo referencias).

Autoría: En qué partes exactas se involucrado cada uno de los 3 miembros del equipo.

Auditoría de Bugs: Transparencia total sobre cualquier funcionalidad que falle.

Declaración de IA: Documentación precisa de qué LLM se usó, en qué partes exactamente, y justificación.

NotebookLM

Desglose de Puntuación

Parte Básica (1.0 pt) - Arquitectura <set>/<map>, robustez, y defensa.

Parte Opcional (0.5 pts) - Innovación justificada con nuevas estructuras.

1.5 Puntos Máximos

El diseño magistral no es solo código que funciona; es código que escala, se explica a sí mismo, y se defiende con rigor.

NotebookLM

Evaluación vs. Ética: Recompensas y Riesgos

Sistema de Evaluación (Máximo 1.5 Puntos)



- **1.0 Pts:** Parte básica perfecta (estructuras obligatorias).
- **0.5 Pts:** Parte opcional (estructuras nuevas, excelente justificación).
- **Requisito:** La nota es idéntica para los 3 miembros, pero exige defensa oral conjunta.

Normativa Ética Inquebrantable

Uso de LLMs: Permitido, pero si se detecta uso no documentado o injustificado, la nota máxima de la práctica se hunde a 0.25 puntos.

Código de Honor (Anti-Plagio)

Se aplicarán sistemas anti-plagio cruzados.
Consecuencia: La detección de plagio implica 0 puntos globales en las prácticas para todo el equipo.

NotebookLM

Política de Integridad y Uso de LLMs

Estado	Acción	Consecuencia
Uso Responsable	LLM utilizado + Documentado y justificado en el Readme.	Evaluado normalmente.
Uso Oculto	Se detecta uso de LLM + NO está documentado ni justificado.	Puntuación máxima capada a 0.25 puntos.
Fraude Académico	Detección de plagio (código cruzado con otros grupos/grados).	0 Puntos globales en todas las prácticas para todo el equipo.

Nota de Defensa: La práctica final debe ser defendida presencialmente por todo el equipo.

NotebookLM

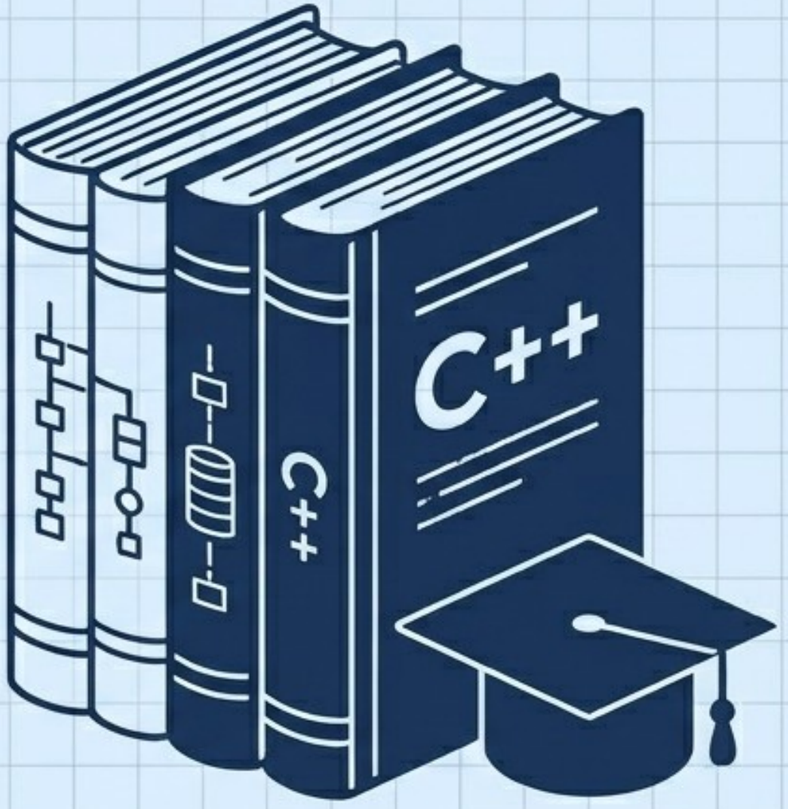
Bibliografía de Referencia y Soporte

Para profundizar en la arquitectura de las estructuras empleadas y dominar la sintaxis avanzada de C++, consulte la siguiente literatura:

Estructuras de Datos en C++. Un enfoque práctico.
Rosa Rodriguez-Sánchez, J. Fdez-Valdivia, J.A. García (2020).

Abstracción y estructuras de datos en C++.
Garrido, A., Fdez-Valdivia, J. Delta publicaciones (2006).

Estructuras de datos avanzadas: con soluciones en C++.
Garrido, A. Editorial Universidad de Granada (2025).



NotebookLM